Acxelin Readiness: How It Works

Oct 29, 2026 · @QubitMan

In one page

Acxelin Readiness finds every piece of cryptography a company depends on, shows what a quantum computer will break, and says what to migrate first.

Every bank runs on hidden cryptography: keys in code, certificates on servers, settings in config files, choices made years ago by people who have left. Before anyone can move to quantum-safe algorithms, someone has to know where all of it is. Most companies cannot answer that today.

The pressure has a date. NIST IR 8547 (draft) deprecates RSA and elliptic-curve cryptography at 112-bit strength after **2030** and disallows it after **2035**. And because attackers can record traffic now and read it once a quantum computer exists (**harvest now, decrypt later**), data that must stay secret for ten years is already exposed.

Acxelin Readiness answers the question in three parts:

Part	What it is, in everyday words	What it does
The scanner	A building inspector with a checklist	Reads a company's code, configuration, keys, certificates, keystores, disk images and network captures, and lists every algorithm it finds, where, and how it was seen.
The endpoint grader	A locksmith testing the front doors	Connects to live TLS and SSH servers and grades each A to F on post-quantum key exchange, weak ciphers and certificates.
The console	The inspector's report, on your own desk	A web page on your own computer that turns findings into four answers: what was found, why it matters, what can happen, and what to do first.

Everything it finds goes into a **CBOM** (Cryptography Bill of Materials) in the CycloneDX 1.6 standard, so auditors and other tools can read it. It runs offline on the customer's own machines and never sends anything to Acxelin Quantum.

The ideas you need

Seven ideas explain the whole product. Each has an everyday picture first, then the technical name.

1. A cryptographic inventory: the list of every lock in the building. Each algorithm, key size and protocol a company uses, and where: a file and line, a certificate, a server. You cannot replace locks you have not found.

2. Two quantum attacks, two different damages. *Shor's algorithm* breaks RSA, elliptic curves and Diffie-Hellman completely. *Grover's algorithm* only halves the strength of symmetric ciphers and hashes, so AES-128 weakens but AES-256 stays safe.

3. Harvest now, decrypt later. Encrypted traffic recorded today can be read once a quantum computer exists. What protects secrets (key exchange, encryption) is urgent now. What proves identity (signatures) matters only once that computer is running.

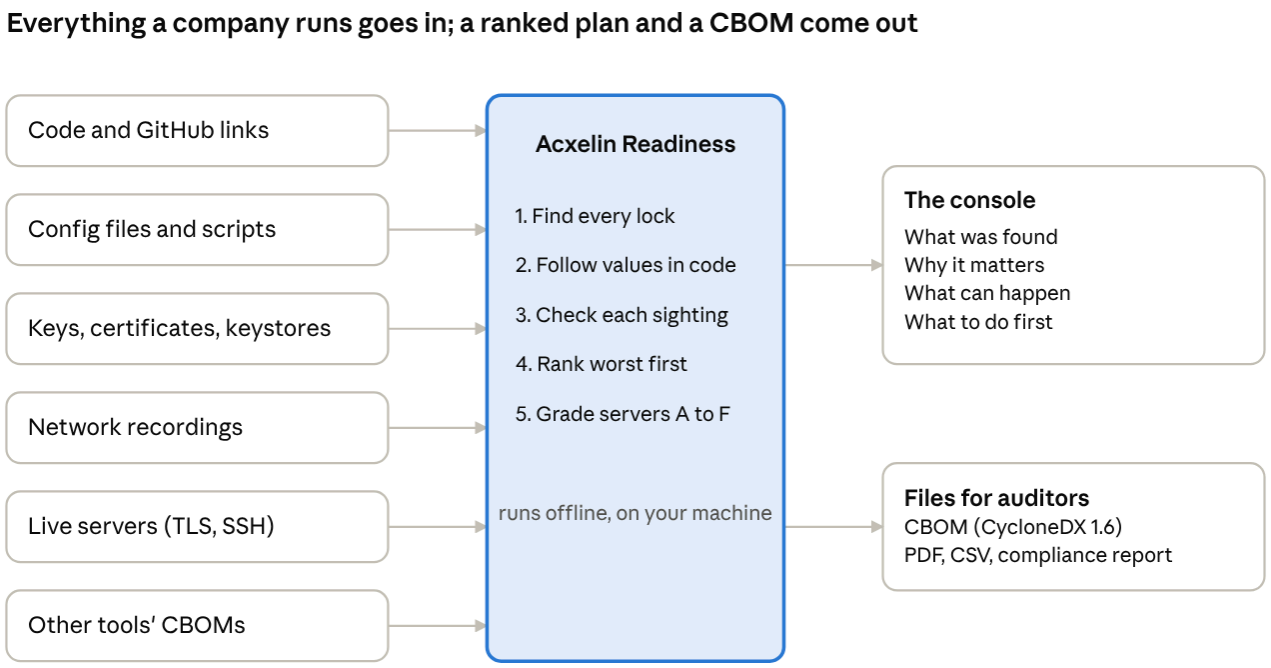
4. Broken today. Some findings need no quantum computer at all: MD5, SHA-1 signatures, RC4, 3DES, DES, TLS 1.0 and 1.1, 1024-bit RSA. They lead every list.

5. Priorities: critical, high, medium, low, ok. Each finding gets one, from what the algorithm is, how exposed its place is (deployed, configured, reachable, or only in tests), and how long the data must stay secret against how long migration takes (Mosca's inequality).

6. The readiness score and its coverage. Each finding counts by its priority (ok 100, low 75, medium 50, high 25, critical 0); the score is their average, 0 to 100. *Coverage* says what share of the files were fully read, so a score never claims more than was checked.

7. The CBOM: the inventory in a standard form. CycloneDX 1.6 is the standard Bill of Materials format; its cryptography part lists each algorithm with its evidence. Auditors, other tools and the next scan all read the same file.

At a glance



Acxelin Readiness · what goes in and what comes out

The scanner reads everything on the left without changing it, decides how weak each lock is, and hands back a plan people can follow and files auditors can check.

Tech stack

Acxelin Readiness is written in Python and needs only one outside library (`cryptography`). It reads code like a person reading a book: it never runs the code, so scanning a client's code is safe.

Part	What we use	Why, in plain words
Language	Python 3.11 to 3.14	Works on Windows, Linux and Mac, and is easy for an auditor to read
Reading code	Our own readers for 14 languages: Python, Java, Kotlin, Go, JavaScript, TypeScript, C, C++, C#, Rust, Swift, PHP, Ruby and shell scripts	Each knows how that language calls cryptography, and can follow a setting from one file to another
Keys and certificates	The `cryptography` library	Opens key files, certificates and keystores to see what is inside

Part	What we use	Why, in plain words
Live servers	Greeting messages we build ourselves, byte by byte	Lets us ask a server about old, weak settings that modern software refuses to offer
Network recordings	Our own reader for Wireshark files	Sees what was really agreed on the wire, without installing Wireshark
Console	One web page, served from your own computer	Nothing extra to install, nothing loaded from the internet
Outputs	Standard file formats (CycloneDX, SARIF, CSV, PDF)	Auditors and other tools can read them without our software
Tests	518 automatic tests, plus a set of sample files that must always be found exactly right	Every change is checked against real code, real recordings and real servers

The code is split in two packages

Package	Lines of Python	Holds
`acxelin-core`	about 2,800	Shared mechanisms: the OpenSSL bridge, the console mechanism, safe file writes. Used by all Acxelin products.
`acxelin-readiness`	about 9,900, plus the console page	The product: detectors, ranking, endpoint grading, the CBOM, reports, the console's routes and page.

The rule between them: **core holds mechanisms, the product holds policy.** Core knows *how* to serve a safe console; Readiness decides *what* counts as weak.

What it reads

One scan reads every place cryptography hides: code, configuration, key files, disk images, network captures, live servers and other tools' CBOMs. Code: `acxelin_readiness/detectors/`.

Source	What it finds	Example
Source code	Calls, constants and names that use an algorithm, with values followed across functions and files	`hashlib.md5(...)`, `Cipher.getInstance("AES/ECB/...")`, `OpenSSL::PKey::RSA.generate(2048)`
Configuration	TLS versions, cipher lists, curves, SSH algorithms, cloud key specs, load-balancer policies	nginx, Apache, HAProxy, `sshd_config`, Kubernetes YAML, Terraform, Java `java.security`
Shell scripts	Key generation and signing on the command line	`openssl genrsa 4096`, `ssh-keygen -t rsa -b 2048`
Keys and certificates	Algorithm, size, curve, signature hash, expiry; private keys flagged as deployed secrets	PEM and DER keys, certificates, CSRs, PKCS#12, Java keystores; trust stores recognised as roots the company trusts, not holds
Dependencies	Crypto libraries and their versions	`requirements.txt, pyproject.toml, package.json, pom.xml, build.gradle, go.mod, Cargo.toml, .csproj`
Binaries and images	Crypto libraries inside built files and container images	a Docker image tarball, a shared library
Encrypted volumes	The cipher protecting a disk	`LUKS 1 and 2, qcow2, BitLocker, encrypted macOS disk images, cry`
Packet captures	What clients and servers actually negotiated	TLS, DTLS, QUIC, SSH, IKEv1 and IKEv2 in `.pcap` and `.pcapng`
Live endpoints	Post-quantum key exchange, weak suites, old TLS versions, certificates	`cloudflare.com`, `ssh://github.com`
Other tools' CBOMs	Their findings, rated on the same scale	a CycloneDX CBOM from CodeQL or another scanner

A GitHub link works as well as a folder: the console downloads a public or private repository (with a token) and scans it the same way.

How a finding is decided

A finding is listed only with its evidence: where it was seen, how sure the scan is, and what it could not see. Here is one finding, step by step.

1. **Spot it.** The scan notes each place a lock appears: a line of code, a setting, a certificate, a server's answer.
2. **Follow it.** When the code chooses the lock through a setting or passes it from one part to another, the scan follows it until it finds the real value. If it cannot prove the value, it says so instead of guessing.
3. **Double-check it.** It looks again at the exact line, so a word in a comment, an option that switches something *off*, or test code is not counted as real use.
4. **Group it.** The same lock in many places becomes one finding with a list of places: RSA-2048 in 14 files is one finding, not 14.
5. **Rank it.** Three things decide how urgent it is: how weak the lock is, how exposed the place is (live on a server, or only in a test), and whether the data it protects must stay secret for longer than the time left before a quantum computer arrives.
6. **Explain it.** Every finding says how it was seen (agreed on a live connection, written in code, set in a config file, or only listed as supported), how sure the scan is (high, medium or low), and what it could not see (for example, the key size).

The five priorities

Priority	Means	Example
critical	Broken or disallowed today, and exposed	MD5 in code that runs; TLS 1.0 in a deployed config
high	Broken today in a less exposed place, or breakable by a quantum computer before migration could finish	RSA-2048 key exchange protecting 10-year data
medium	Breakable later, with time left to migrate; or a classical fallback beside a hybrid group	ECDSA signatures; X25519 next to X25519MLKEM768
low	Weakened by quantum search only	AES-128, SHA-224
ok	Quantum-safe, or no known quantum break at this strength	AES-256, SHA-384, ML-KEM, ML-DSA

Guards against false alarms

Each of these came from scanning a real project, and each has a test.

- An old fingerprint method (SHA-1, MD5) used only where faking a fingerprint does not help an attacker is rated low, not broken.
- Code that says outright "this hash is not for security" is rated low.
- Root certificates the computer merely *trusts* are not counted as keys the company *holds*.
- Names that only look alike are not confused: `arc4random` is a random-number source, not the old RC4 cipher.
- Test code is listed separately and does not lower the score.

Grading live servers

Each TLS or SSH server gets a grade from A to F on one question: does its key exchange protect recorded traffic from a future quantum computer? Code: `acxelin_readiness/endpoints.py` and `detectors/probe.py`.

Grade	Means	What to do
A	Post-quantum key exchange only	Nothing on key exchange
B	Post-quantum offered, classical still accepted	Stop accepting classical groups once all clients offer post-quantum
C	Classical only: traffic recorded today can be decrypted later	Enable TLS 1.3 with X25519MLKEM768 (or ML-KEM for SSH)
F	Unreachable, or no TLS or SSH	Check the address, port and firewall

What one check does

1. **Greet it like a browser.** It connects the way Chrome does, offering the new quantum-safe lock and the classic one together, and notes which one the server picks.
2. **Ask about every lock, one at a time.** It asks the server about each quantum-safe and each classic option. These questions are built by hand, so the answer does not depend on how new the scanning computer's own software is.
3. **Ask about old and broken settings.** Does it still accept TLS 1.0 or 1.1? RC4, 3DES, or Diffie-Hellman keys under 2048 bits? Each one found is named.

4. **Read its ID card.** The certificate's key type, who issued it, how many days are left, and whether this computer trusts it. If every server shows the same issuer, the page warns that a company proxy probably answered instead.
5. **SSH servers too.** Which quantum-safe options it offers for remote logins, and whether it still offers a DSA key, which OpenSSH has removed.

A check that gets no answer says *could not be checked*, never *none found*. Real servers graded this way in October 2026: Cloudflare, Google, Microsoft, AWS and Apple got B; GitHub's website got C; GitHub's SSH got B.

The console

The console is a web page that runs on your own computer, and only your computer can open it. You point it at your code or your servers and it tells you, in plain words, what is weak and what to fix first. No typing commands.

Opening it

1. Open PowerShell (Windows) or a terminal (Linux, macOS).
2. Type `acxelin-readiness console` and press Enter.
3. Your browser opens at `http://127.0.0.1:8900`, already signed in. Keep the window open; press **Ctrl+C** to stop.

If the browser does not open, add `--no-browser` and copy the address it prints. Each address works once; `acxelin-readiness console link` gives a fresh one.

The two pages

Page	What you do there
Projects	Choose a folder of code, paste a GitHub link (public, or private with a token), or load a CBOM from another tool. Press Scan.
Endpoints	Paste server addresses, one per line (`bank.example.com`, `ssh://server`). Press Check.

What every result shows

Each result opens with a short summary at the top, then four sections that each answer one question:

Section	The question it answers	What you see
What was found	What cryptography do we have?	A colour bar of priorities, the list of findings, and a progress bar of what is already fixed
Why	Why does each one matter?	What each priority means, what puts the findings at risk, how sure the scan is, the score over time
What can happen	What happens if we do nothing?	Plain risks, such as "traffic recorded today can be read later" or "this certificate expires in 15 days"
What to do	What should we fix first?	Numbered steps, worst first, each naming the findings it fixes

Keeping track of the fixes

- **Owner and due date** for each finding. The console suggests an owner from the code's ownership file (CODEOWNERS) or from a spreadsheet of who owns what.
- **Status:** To do, Planned, In progress, Fixed, or Accepted risk (with a reason and an end date). A finding marked Fixed that the next scan still finds is flagged.
- **Downloads:** a spreadsheet (CSV), a PDF (Print), and the CBOM for auditors.

How it is kept safe

Protection	In plain words
Only this computer	It refuses anyone who is not on your own machine.
A one-time link	The address works once, and the secret part never reaches a server log.
No cookies	Another website you visit cannot act through your open console.
Lock-out	Ten wrong tries in a minute locks that sender out for the minute.
No outside scripts	The page runs only its own code, checked by a fingerprint.
Private tokens stay private	A GitHub token is stored only for your user and never shown again.

What auditors and risk teams get

Every scan leaves files an auditor can read without our software. All use open, standard formats.

File	In plain words	Who reads it
CBOM (`cbom.json`)	The full list of cryptography, with where each item was found, in the CycloneDX 1.6 standard	Auditors, risk tools, the next scan (to show what is new)
Report (HTML, printable to PDF)	A readable summary: priorities, risks, what to fix first	Managers and security leaders
Spreadsheet (CSV)	One row per finding with owner, status and due date	The team doing the fixes
Compliance report (`acxelin-readiness report`)	Which certificates, servers and findings break the US deadlines in NIST IR 8547 (2030 and 2035) or the NSA's CNSA 2.0 list	Compliance and audit teams

File	In plain words	Who reads it
Code scanning results (SARIF)	Findings in the format GitHub and other code tools show next to the code	Developers
Company-wide view (`acxelin-readiness merge`)	Many scans (repositories, servers, other tools' CBOMs) combined into one inventory and dashboard	Whoever owns the migration programme

A scan can also act as a gate in a build pipeline: compared with the last CBOM, it fails the build only when something new and weak appears.

Code tour

Read it in the order a scan runs: the pipeline, the detectors, the checks, the ranking, then the outputs and the console. Paths below are under `products/readiness/src/acxelin_readiness/`.

The scan, in order

File	Lines	What it does
`pipeline.py`	175	The running order: every detector, then the checks, then ranking. Start here.
`detectors/source.py`, `rules.py`, `pysrc.py`	295, 587, 342	Reading code. `rules.py` holds one short rule per library call, per language; `pysrc.py` reads Python properly through its syntax tree.
`detectors/pyflow.py`	457	Follows a value through Python: a setting in one file used in another.
`detectors/config.py`, `artifacts.py`, `keystores.py`	178, 233, 137	Config files, key and certificate files, Java and PKCS#12 keystores.
`detectors/capture.py`, `quic.py`, `ike.py`	448, 101, 116	Reading handshakes out of network captures.
`verify.py`	141	Checks each sighting against its line, so comments and switched-off options do not count.

File	Lines	What it does
`rank.py`, `catalog.py`	389, 322	The priority of each finding, and the list of every algorithm with what breaks it and what replaces it.
`endpoints.py`, `detectors/probe.py`	419, 219	Grading live servers, including the hand-built hello messages.
`cbom.py`, `report.py`, `evidence.py`	222, 225, 225	The CBOM, the HTML report, the CSV and the compliance mapping.
`workspace.py`, `console.py` + `console.html`	416, 414 + page	The console: projects, tracking, owners, and its 17 routes.

Design decisions worth discussing

- **Why read code and never run it?** Running a client's code to see what it does would be unsafe and slow. Reading it is safe on any machine.
- **Why build hello messages by hand?** To test whether a server still accepts TLS 1.0, RC4 or 3DES, the scanner must offer them, and modern OpenSSL refuses to. Hand-built bytes work everywhere.
- **Why say "could not check" instead of guessing?** A wrong "all clear" is worse than an honest gap; coverage is shown next to every score.
- **How do we know it is accurate?** A labelled set of sample files must be found with precision and recall of 1.000 on every change (179 labelled pairs); changing the labels needs a separate, approved commit. Every false alarm found on real projects became a test.

Installing and releases

A customer downloads one file, checks it has not been changed, and runs one installer; no internet is needed after the download.

What a customer gets

The current release is **1.1.0**, tag `readiness-v1.1.0` on GitHub, published 29 October 2026. It has two files:

- `acxelin-readiness-1.1.0-wheelhouse.tar.gz`: the program and everything it needs, for Linux, Windows and Apple-silicon Macs, with the installers, the manual and the licence inside.

- `SHA256SUMS`: the fingerprint of the download, to prove it arrived unchanged.

There is also a ready-made container image, `ghcr.io/qubitman-hub/acxelin-readiness:1.1.0`.

Installing

1. Check the fingerprint: `sha256sum -c SHA256SUMS --ignore-missing` (on Windows, `Get-FileHash`).
2. Unpack: `tar xzf acxelin-readiness-1.1.0-wheelhouse.tar.gz`, then go into the folder.
3. Run `sh install.sh` (Linux, macOS) or `install.ps1` (Windows). It checks Python, adds the `acxelin-readiness` command and runs a self-check (`doctor`).
4. First scan: `acxelin-readiness scan path/to/project`, or open the console.

How a release is made

The owner pushes a tag such as `readiness-v1.1.0`. GitHub then checks the version matches, runs every test, builds the download for every platform, builds the container image, writes the fingerprints, attaches proof that our workflow built the file, and publishes the release. A product release needs a released core first; 1.1.0 of both went out together.

Use cases

Readiness answers four questions a bank is asked, all things the product does today.

1. "What cryptography do we have?"

Use case	Example
Inventory of one application	Scan the payments service's code before a migration project starts
Inventory across the company	Scan many repositories and servers, then merge them into one list
What a vendor ships	Scan a container image or a vendor's CBOM before accepting their product
Keys and certificates lying around	Find private keys committed to code, and certificates about to expire

2. "What is exposed to quantum attack today?"

Use case	Example
Recorded traffic	Which connections use key exchange a quantum computer could break later
Public and partner servers	Grade the bank's websites, APIs and SSH servers A to F
What really happens on the wire	Read a network capture to see what clients and servers actually agreed on

3. "What do we fix first?"

Use case	Example
A migration plan	The ranked list, worst first, each with its replacement (for example ML-KEM-768 instead of RSA)
Owners and deadlines	Each finding gets an owner from CODEOWNERS or the company's ownership spreadsheet, a status and a due date
Proof that fixes landed	The next scan checks every finding marked Fixed

4. "Can we prove it to the regulator?"

Use case	Example
Evidence for auditors	A standard CBOM and a compliance report against NIST IR 8547 and CNSA 2.0
Progress over time	The readiness score and coverage after every scan
Stopping new weak code	A build check that fails only when something new and weak is added

What it doesn't cover

- **Fixing the code.** It says what to change and to what; people make the change.
- **Code it cannot see.** Encrypted or compiled-only code is read only as far as the libraries inside it.

Explaining it to a client

Start with the deadline and their own data, show their own result, then answer the questions every client asks.

A five-step talk track

1. **The risk, in their terms.** "Regulators now expect a plan to move off RSA and elliptic curves: NIST's draft plan phases them out after 2030 and bans them after 2035. Your account data must stay secret for ten years, so anyone recording your traffic today could read it later."
2. **The first step.** "You cannot replace what you have not found. Readiness finds the locks in your systems without changing anything."
3. **Show it.** Paste one of their public addresses into the Endpoints page: they see their own grade in seconds, next to Cloudflare's B.
4. **Show the plan.** Scan one of their repositories (or a public sample): the console shows what was found, why it matters, what can happen and what to fix first.
5. **The path.** Assign owners, fix the critical items, scan again: the score rises and the auditor gets a standard CBOM.

Questions clients ask

Question	Answer
Does our code have to leave our building?	No. It runs on their own machine, offline; nothing is sent to us.
Does it change anything?	No. It only reads code and connects to servers the way any client would.
Will it flood us with false alarms?	It shows its evidence and how sure it is for every finding, and every false alarm found on real projects became a test.
Is it a standard format?	Yes: CycloneDX 1.6 for the inventory, SARIF for code tools, CSV and PDF for people.
What about code you cannot read?	The score shows its coverage, so a gap is visible, never hidden.
What if your company disappears?	Everything it writes is in open formats, and it runs from the files they hold.

Limits: what we do not claim

Say these before the client finds them; it builds more trust than any feature.

Limit	What it means in practice
It finds what it can see	Code in languages it does not read, encrypted files, and cryptography inside closed-source programs may be missed. Coverage shows how much was read.
Values it cannot prove stay unknown	When code picks an algorithm at run time from outside the code (a database, a remote setting), the finding says the size or mode was not seen.
Live grades describe what answered	A company proxy that inspects TLS answers for every server; the page warns when every host shows the same issuer.
The browser-like check is newly changed	Offering two key shares, as Chrome does, was added in 1.1.0 and is not yet confirmed against real servers; older OpenSSL falls back to the plain check.
Revocation is not checked	A certificate cancelled by its issuer can still read as trusted.
Very large Python projects	Above 5,000 Python files, values are followed only one file away from cryptography code, to keep memory low; the scan says so.
Priorities are a model	They follow the NIST dates and Mosca's inequality with default numbers (10 years of secrecy, 5 of migration, a quantum computer in 9); a client can set their own.
Not a certified product	It uses public standards, but has no FIPS or Common Criteria certificate of its own.

Glossary

Term	In everyday words
3DES, DES, RC4	Old ways of scrambling data, breakable today without any quantum computer.

Term	In everyday words
AES	Today's standard way to scramble data. AES-256 stays safe against quantum computers; AES-128 is weakened.
CBOM	Cryptography Bill of Materials: the list of every lock a system uses, in a standard file.
Certificate	A computer's ID card: its name, its public key, and a signature from someone others trust.
CNSA 2.0	The US National Security Agency's list of algorithms allowed for secret systems.
Coverage	The share of files the scan fully read. A score is only as good as its coverage.
CycloneDX	An open standard for listing what a piece of software contains.
Diffie-Hellman (DH, ECDH)	Ways two computers agree a secret in public. Breakable by a large quantum computer.
Grover's algorithm	A quantum method that halves the strength of locks like AES: AES-256 behaves like AES-128.
Harvest now, decrypt later	Recording encrypted traffic today to read it once a quantum computer exists.
Hash (MD5, SHA-1, SHA-256)	A fingerprint of data. MD5 and SHA-1 can be faked today; SHA-256 is fine.
HMAC	A hash with a secret key, used to prove a message was not changed.
Hybrid key exchange	Two locks on one door: a classical one plus a quantum-safe one.
Keystore	A file that holds keys and certificates, such as a Java keystore or a PKCS#12 file.
ML-DSA	The new quantum-safe signature standard (FIPS 204).
ML-KEM	The new quantum-safe way to agree a secret (FIPS 203).

Term	In everyday words
Mosca's inequality	If secrets must last X years and moving takes Y years, you are late when X + Y is more than the years until a quantum computer.
NIST IR 8547	The US standards body's draft plan: phase out RSA and elliptic curves after 2030, ban them after 2035.
Packet capture	A recording of network traffic, such as a Wireshark file.
Readiness score	0 to 100: the average of all findings, where ok counts 100 and critical counts 0.
RSA, ECDSA, Ed25519	Today's signature and key methods. Breakable by a large quantum computer.
Shor's algorithm	The quantum method that breaks RSA and elliptic curves completely.
TLS, SSH	The protocols that protect web traffic and remote logins. TLS 1.0 and 1.1 are retired; 1.3 is current.
X25519MLKEM768	The hybrid key exchange Chrome, Cloudflare and Google use today.